



山东大学

信息科学与工程学院

XXX—XXX 学年第二学期

课程实验报告

课程名称: 数据结构与算法导论

实验名称: ADT 表的编程与实现

专 业 班 级 通信二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2026 年 5 月 13 日

一、实验目的

1. 加深对抽象数据类型 ADT 表的理解；

二、实验原理

栈：

栈是一种操作受限制的线性表，它只允许在表的一端进行插入和删除操作，这一端称为**栈顶**，另一端称为**栈底**。向栈中加入元素的操作称为**入栈**或**压栈**，英文为 **push**；从栈顶删除元素的操作称为**出栈**或**弹栈**，英文为 **pop**。

由于栈中只能访问和操作栈顶元素，所以最先进入栈的元素要最后才能被取出，最后进入栈的元素会最先被取出。因此，栈具有**后进先出**的特点，即 **LIFO, Last In First Out**。

栈可以理解为一个底部封闭、顶部打开的容器，元素只能从顶部放入，也只能从顶部取出。例如向栈中依次压入元素 A、B、C，则 C 位于栈顶，出栈时的顺序为 C、B、A。

栈的常见基本操作包括：

push：将元素压入栈顶

pop：删除并返回栈顶元素

top / getTop：获取栈顶元素但不删除

isEmpty：判断栈是否为空

在程序设计中，栈常用于函数调用、递归实现、表达式求值、括号匹配以及进制转换等场景。栈既可以用数组实现，也可以用链表实现。用数组实现时需要注意栈满问题，用链表实现时则可以根据需要动态申请空间。**队列**

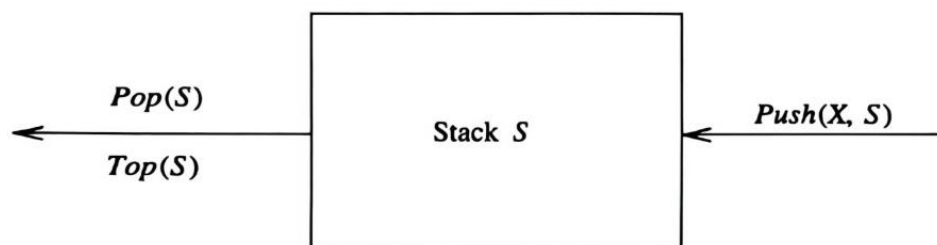


图 3-37 栈模型：通过 Push 向栈输入，通过 Pop 从栈输出

队列也是一种操作受限制的线性表。与栈不同，队列的插入和删除操作分别在两端进行。队列只允许在一端插入元素，这一端称为**队尾**；只允许在另一端删除元素，这一端称为**队头**。向队列中加入元素的操作称为**入队**，从队列中删除元素的操作称为**出队**。

队列的特点是最先进入队列的元素最先被删除，最后进入队列的元素最后被删除。因此，队列具有**先进先出**的特点，即 **FIFO, First In First Out**。

队列可以理解为日常生活中的排队过程，先来的人排在前面并先离开，后来的人从队尾加入并后离开。例如向队列中依次加入元素 A、B、C，则 A 位于队头，C 位于队尾，出队顺序为 A、B、C。

队列的常见基本操作包括：

enqueue：元素入队，加入队尾

dequeue：元素出队，删除队头元素

front / getFront: 获取队头元素但不删除

isEmpty: 判断队列是否为空

队列也可以用数组或链表实现。使用数组实现普通队列时，随着出队操作进行，数组前面的空间可能被浪费，因此实际应用中常使用**循环队列**来提高空间利用率。使用链表实现队列时，通常设置头指针和尾指针，分别指向队头和队尾，便于进行入队和出队操作。

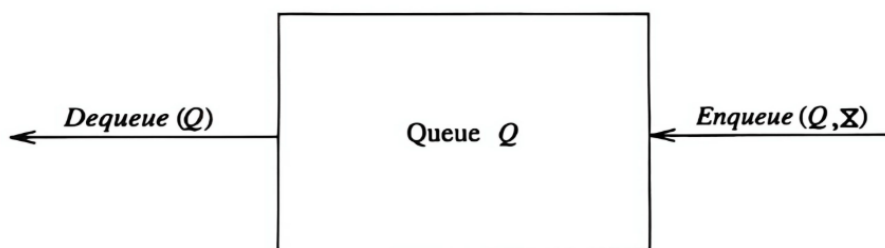


图 3-56 队列模型

三、实验内容

1. 编写程序实现 ADT 栈的定义，及常用操作（数组或指针实现）：

- 1) 生成栈
- 2) Push
- 3) Pop

2. 编写程序实现 ADT 队列的定义，及常用操作：

- 1) 生成队列
- 2) Enqueues 入列
- 3) Isempty 判断是否队列为空

四、实验要求

- 1) 实现 ADT 栈的结构及操作
- 2) 实现 ADT 队列的结构及操作，并给出应用

五、实验源程序

1. 实现 ADT 栈的结构及操作

```
#include <stdio.h>
#include <stdlib.h>

#define MIN_STACK_SIZE 5
#define EMPTY_TOP -1
```

```
// 定义栈结构体
typedef struct StackNode
{
    int capacity;    // 栈的最大容量
    int top;         // 栈顶下标
    int *data;       // 用数组存储栈元素
} *Stack;

// 函数声明
Stack InitStack(int size);
int IsEmpty(Stack S);
int IsFull(Stack S);
void Push(Stack S, int x);
void Pop(Stack S);
void PrintStack(Stack S);
void ClearStack(Stack S);
void DestroyStack(Stack S);

int main()
{
    int size;
    Stack S = NULL;

    // 创建栈
    while (S == NULL)
    {
        printf("请输入栈的容量: ");
        scanf("%d", &size);

        S = InitStack(size);

        if (S == NULL)
        {
            printf("栈创建失败, 请重新输入容量! \n");
        }
    }

    printf("栈创建成功! \n");

    int choice;
    int value;

    while (1)
    {
```

```

printf("\n===== 栈的基本操作 =====\n");
printf("1. Push 入栈\n");
printf("2. Pop 出栈\n");
printf("3. 显示栈中元素\n");
printf("0. 退出程序\n");
printf("请选择操作: ");
scanf("%d", &choice);

switch (choice)
{
case 1:
    printf("请输入要入栈的数据: ");
    scanf("%d", &value);
    Push(S, value);
    PrintStack(S);
    break;

case 2:
    Pop(S);
    PrintStack(S);
    break;

case 3:
    PrintStack(S);
    break;

case 0:
    DestroyStack(S);
    printf("程序结束, 栈空间已释放.\n");
    return 0;

default:
    printf("输入有误, 请重新选择! \n");
}
}

// 生成栈
Stack InitStack(int size)
{
    Stack S;

    // 判断输入容量是否合法
    if (size < MIN_STACK_SIZE)

```

```

{
    printf("栈容量不能小于 %d! \n", MIN_STACK_SIZE);
    return NULL;
}

// 给栈结构体分配内存
S = (Stack)malloc(sizeof(struct StackNode));
if (S == NULL)
{
    printf("结构体空间申请失败! \n");
    return NULL;
}

// 给数组分配内存
S->data = (int *)malloc(sizeof(int) * size);
if (S->data == NULL)
{
    printf("数组空间申请失败! \n");
    free(S);
    return NULL;
}

S->capacity = size;
S->top = EMPTY_TOP;

return S;
}

// 判断栈是否为空
int IsEmpty(Stack S)
{
    return S->top == EMPTY_TOP;
}

// 判断栈是否已满
int IsFull(Stack S)
{
    return S->top == S->capacity - 1;
}

// Push: 入栈操作
void Push(Stack S, int x)
{
    if (IsFull(S))

```

```

    {
        printf("栈已满，无法继续入栈！\n");
        return;
    }

    S->top++;
    S->data[S->top] = x;

    printf("元素 %d 入栈成功.\n", x);
}

// Pop: 出栈操作
void Pop(Stack S)
{
    if (IsEmpty(S))
    {
        printf("当前栈为空，无法出栈！\n");
        return;
    }

    printf("出栈元素为: %d\n", S->data[S->top]);
    S->top--;
}

// 输出栈中元素
void PrintStack(Stack S)
{
    if (IsEmpty(S))
    {
        printf("当前栈为空.\n");
        return;
    }

    printf("当前栈中元素为: ");

    for (int i = 0; i <= S->top; i++)
    {
        printf("%d ", S->data[i]);
    }

    printf("\n");
}

// 清空栈

```

```
void ClearStack(Stack S)
{
    S->top = EMPTY_TOP;
}

// 销毁栈，释放内存
void DestroyStack(Stack S)
{
    if (S != NULL)
    {
        free(S->data);
        free(S);
    }
}
```

六、实验结果

1. 实现 ADT 栈的结构及操作

- 请输入栈的容量：5
栈创建成功！

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：1
请输入要入栈的数据：1
元素 1 入栈成功。
当前栈中元素为：1

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：1
请输入要入栈的数据：2
元素 2 入栈成功。
当前栈中元素为：1 2

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈

===== 栈的基本操作 =====

1. Push 入栈
 2. Pop 出栈
 3. 显示栈中元素
 0. 退出程序
 请选择操作：1
 请输入要入栈的数据：3
 元素 3 入栈成功。
 当前栈中元素为：1 2 3

===== 栈的基本操作 =====

1. Push 入栈
 2. Pop 出栈
 3. 显示栈中元素
 0. 退出程序
 请选择操作：1
 请输入要入栈的数据：4
 元素 4 入栈成功。
 当前栈中元素为：1 2 3 4

===== 栈的基本操作 =====

1. Push 入栈
 2. Pop 出栈
 3. 显示栈中元素
 0. 退出程序
 请选择操作：1
 请输入要入栈的数据：5
 元素 5 入栈成功。
 当前栈中元素为：1 2 3 4 5

===== 栈的基本操作 =====

1. Push 入栈
 2. Pop 出栈
 3. 显示栈中元素
 0. 退出程序
 请选择操作：1
 请输入要入栈的数据：6
 栈已满，无法继续入栈！
 当前栈中元素为：1 2 3 4 5

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：2

出栈元素为：5

当前栈中元素为：1 2 3 4

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：2

出栈元素为：4

当前栈中元素为：1 2 3

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：2

出栈元素为：3

当前栈中元素为：1 2

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：█

当前栈中元素为：1 2 3

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：2

出栈元素为：3

当前栈中元素为：1 2

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：3

当前栈中元素为：1 2

===== 栈的基本操作 =====

1. Push 入栈
2. Pop 出栈
3. 显示栈中元素
0. 退出程序

请选择操作：0

程序结束，栈空间已释放。

○ PS D:\Fnan\Studv\数据结构\源程序> █

2. 实现 ADT 队列的结构及操作

0. 退出程序

请选择操作：1

请输入要入队的数据：55

元素 55 入队成功。

当前队列中的元素为：15 55

=====

1. 入队

2. 出队

3. 显示队列

0. 退出程序

请选择操作：1

请输入要入队的数据：14

元素 14 入队成功。

当前队列中的元素为：15 55 14

=====

1. 入队

2. 出队

3. 显示队列

0. 退出程序

请选择操作：1

请输入要入队的数据：19

元素 19 入队成功。

当前队列中的元素为：15 55 14 19

=====

1. 入队

2. 出队

3. 显示队列

0. 退出程序

请选择操作：1

请输入要入队的数据：19

队列已满，无法入队！

当前队列中的元素为：15 55 14 19

=====

```

=====
1. 入队
2. 出队
3. 显示队列
0. 退出程序
请选择操作：2
出队元素为：15
当前队列中的元素为：55 14 19
    
```

```

=====
1. 入队
2. 出队
3. 显示队列
0. 退出程序
请选择操作：2
出队元素为：55
当前队列中的元素为：14 19
    
```

```

=====
1. 入队
2. 出队
3. 显示队列
0. 退出程序
请选择操作：2
出队元素为：14
当前队列中的元素为：19
    
```

```

=====
1. 入队
2. 出队
3. 显示队列
0. 退出程序
请选择操作：
    
```

当前队列中的元素为：19

```

=====
1. 入队
2. 出队
3. 显示队列
0. 退出程序
请选择操作：3
当前队列中的元素为：19
    
```

```

=====
1. 入队
2. 出队
3. 显示队列
0. 退出程序
请选择操作：0
程序结束，队列空间已释放。
    
```

PS D:\Epan\Study\数据结构\源程序> █

七、实验心得

本次实验主要完成了栈和队列两种线性数据结构的基本操作实现。通过编写程序，我进一步理解了栈和队列虽然都属于线性表，但它们在数据插入和删除规

则上存在明显区别。

在栈的实验中，程序采用数组作为存储结构，通过结构体记录栈的容量、栈顶位置以及存放数据的数组。栈的核心操作包括生成栈、入栈和出栈。入栈时，先判断栈是否已满，若未满则将栈顶指针上移并存入新元素；出栈时，先判断栈是否为空，若非空则输出栈顶元素并将栈顶指针下移。通过实验可以看出，栈只能在栈顶进行插入和删除操作，因此具有“后进先出”的特点。

在队列的实验中，程序采用数组实现循环队列，通过结构体记录队列容量、队头位置、队尾位置和当前元素个数。队列的基本操作包括生成队列、入队和出队。入队操作在队尾进行，出队操作在队头进行。为了避免数组前端空间被浪费，程序使用取模运算使队头和队尾指针在数组中循环移动，从而实现循环队列。实验结果表明，队列中的元素按照进入顺序依次离开，体现了“先进先出”的特点。

通过本次实验，我加深了对 ADT 栈和 ADT 队列的理解。ADT 更强调数据结构的逻辑功能和基本操作，而数组只是其中一种具体的存储实现方式。在程序实现过程中，还需要注意边界条件的判断，例如栈满、栈空、队满和队空等情况。如果不进行这些判断，程序可能会出现数组越界或错误删除数据的问题。因此，本次实验不仅帮助我掌握了栈和队列的基本操作，也提高了我对数据结构抽象定义和具体实现之间关系的理解。